

Воспроизведение EAGLE-3 на LLaMA-3.1-8B-Instruct

Ускорение кодом авторов, скорость обучения: код авторов против NeMo.
Промежуточный отчёт

Смена по воспроизведению EAGLE-3

9 октября 2026

Коротко

1. **Статья воспроизводится по τ .** Официальный драфт и код авторов дают τ 6,25 против 6,23 в статье ($T = 0$) и 4,91 против 4,92 ($T = 1$) — совпадение в пределах 1 %.
2. **Ускорение ниже статьи:** 3,38× против 4,44× ($T = 0$), 2,72× против 3,48× ($T = 1$). Драфт тут ни при чём: на H200 код авторов упирается в запуск тысяч мелких CUDA-ядер с CPU, и цикл EAGLE стоит 1,85 обычного шага вместо 1,4.
3. **В коде авторов три ловушки:** глубина дерева по умолчанию от EAGLE-2, лимит ответа 1024 не передаётся (фактически 512), опубликованный тренер падает при запуске.
4. **NeMo обучает драфт быстрее кода авторов всего в 1,06–1,24×** (каждый в своих настройках): главный приём NeMo — упаковка примеров — даёт то же, что микро-батч 1 без паддинга у авторов. На одинаковой работе NeMo быстрее в 1,38×.
5. **В работе:** SGLang (задача 3b) и бюджет GPU-часов (задача 5).

Что и как мерили

Tapret — LLaMA-3.1-8B-Instruct (fp16), драфт — официальный yuhuili/EAGLE3-LLaMA3.1-Instruct-8B, одна NVIDIA H200 NVL. Код авторов (SafeAILab/EAGLE, коммит cb7e084) — без изменений, вопросы — из их репозитория (5 бенчмарков по 80), формула ускорения — из их speed.py. Дерево как в статье EAGLE-3: 60 токенов, глубина 8, top-k 10. τ — среднее число токенов за один проход большой модели.

Задача	Что	Статус
3a	ускорение кодом авторов	готово
3b	то же в SGLang (движок, которым NeMo обслуживает драфты)	идёт прогон
4	скорость шага обучения: код авторов против NeMo AutoModel	готово
5	стоимость регенерации данных и бюджет GPU-часов против 100 ч	идёт прогон

За. Код авторов: τ совпадает, ускорение ниже

Бенчмарк	T = 0: ускорение / τ		T = 1: ускорение / τ	
	статья	у нас	статья	у нас
MT-bench	4,40× / 6,13	3,29× / 6,18	3,07× / 4,24	2,34× / 4,12
HumanEval	4,85× / 6,74	3,74× / 6,74	4,13× / 5,82	3,17× / 5,81
GSM8K	4,48× / 6,23	3,46× / 6,26	3,32× / 4,59	2,98× / 4,84
Alpaca	4,82× / 6,70	3,68× / 6,70	3,90× / 5,56	2,91× / 5,29
CNN/DM	3,65× / 5,34	2,75× / 5,39	2,99× / 4,39	2,19× / 4,48
Среднее	4,44× / 6,23	3,38× / 6,25	3,48× / 4,92	2,72× / 4,91

За. τ совпадает со статьёй, ускорение ниже

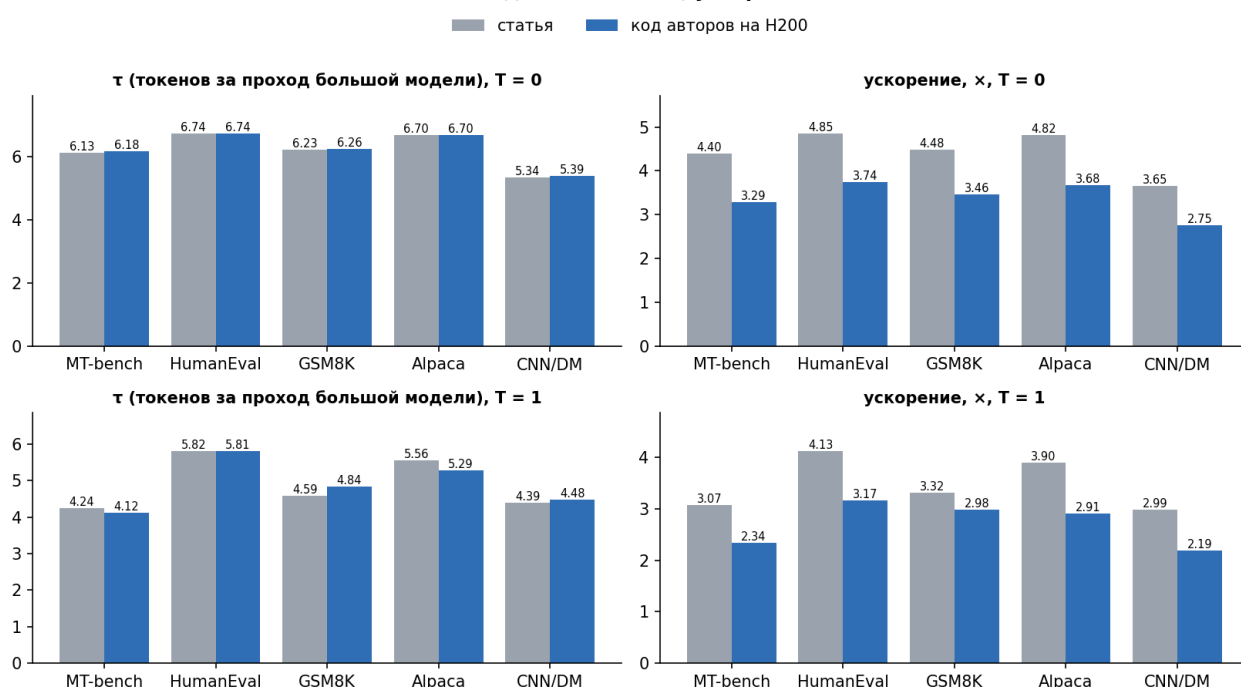


Рис. 1: τ и ускорение по бенчмаркам: статья и код авторов на H200 (глубина дерева 8).

Почему ускорение ниже при том же τ . Ускорение = τ / (стоимость цикла EAGLE в шагах обычной генерации). У нас цикл стоит 1,85 обычного шага, у авторов — 1,40. Профиль (MT-bench, 5 вопросов) показывает, откуда это:

	Время, мс	Запусков CUDA-ядер	GPU реально считает
Шаг обычной генерации	12,9	1 819	8,4 мс ($\approx 65\%$)
Цикл EAGLE	21,5	2 967	15,6 мс ($\approx 73\%$)
проверка дерева большой моделью	13,6		
драфт: 8 шагов дерева	6,9		
выбор ветки, обновление, Python, prefill	1,0		

- Проверка 60 токенов дерева стоит почти как один обычный шаг (13,6 против 12,9 мс) — как и задумано.
- Лишнее время цикла — драфт: 6,9 мс $\approx 0,54$ обычного шага. Драфт из одного слоя делает

8 последовательных шагов из сотен мелких ядер, и время уходит не на вычисления, а на их запуск с CPU.

- **Обе генерации ограничены запуском ядер, а не GPU:** нижняя граница шага по памяти для 16 ГБ весов — 3,3 мс, а шаг занимает 12,9 мс. Отношение числа запусков ($2967 / 1819 = 1,63$) почти равно отношению времени ($21,5 / 12,9 = 1,67$).
- На H200 с быстрой памятью обычный шаг относительно дешевле, а накладные расходы драфта не уменьшаются — отсюда и цикл дороже, чем у авторов (GPU для Table 1 в статье не указан). Лечится CUDA graphs / torch.compile — именно это проверяем в SGLang (задача 3b).

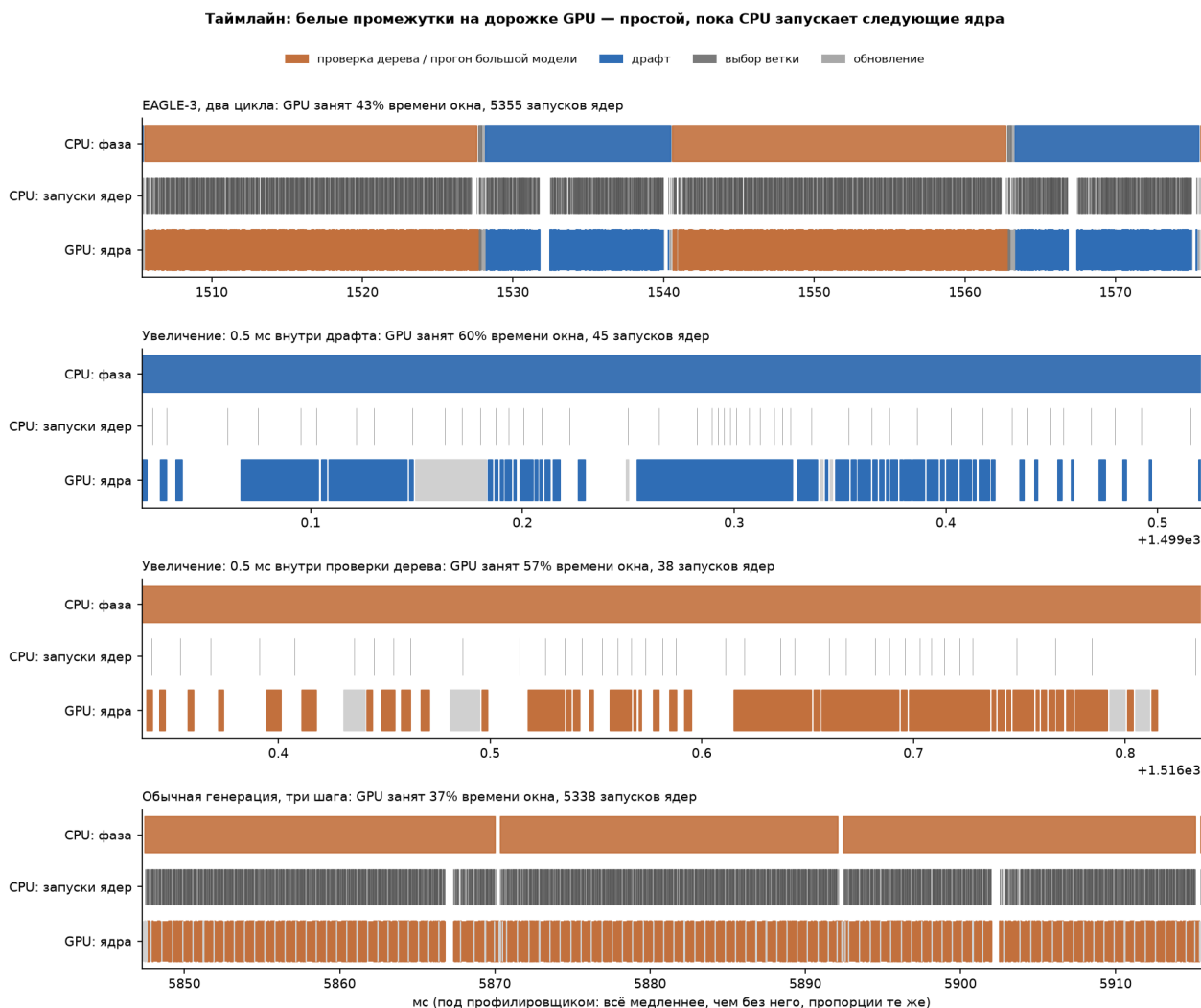


Рис. 2: Таймлайн по torch.profiler: два цикла EAGLE (проверка дерева — оранжевым, драфт — синим), увеличенные окна по 0,5 мс внутри драфта и внутри проверки, и три шага обычной генерации. Белые промежутки на дорожке GPU — простой, пока CPU запускает следующее ядро. Под профилировщиком CPU медленнее, поэтому доли простоя здесь больше, чем без него (без профилировщика GPU занят $\approx 65\text{--}73\%$), но картина та же.

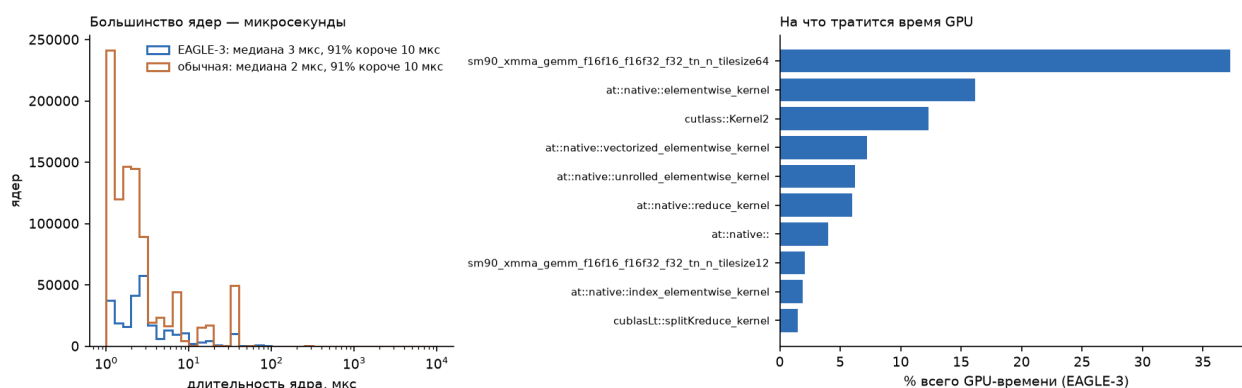


Рис. 3: Слева: 91 % запусков ядер короче 10 мкс (медиана 2–3 мкс) — время уходит на их запуск, а не на вычисления. Справа: на что тратится время GPU в EAGLE: GEMM-ядра и мелкие поэлементные операции.

Глубина дерева важна. Скрипт оценки авторов по умолчанию строит дерево EAGLE-2 (глубина 6). С ним τ на 10 % ниже статьи (5,55 против 6,23, $T = 0$), и легко сделать вывод «статья не воспроизводится».

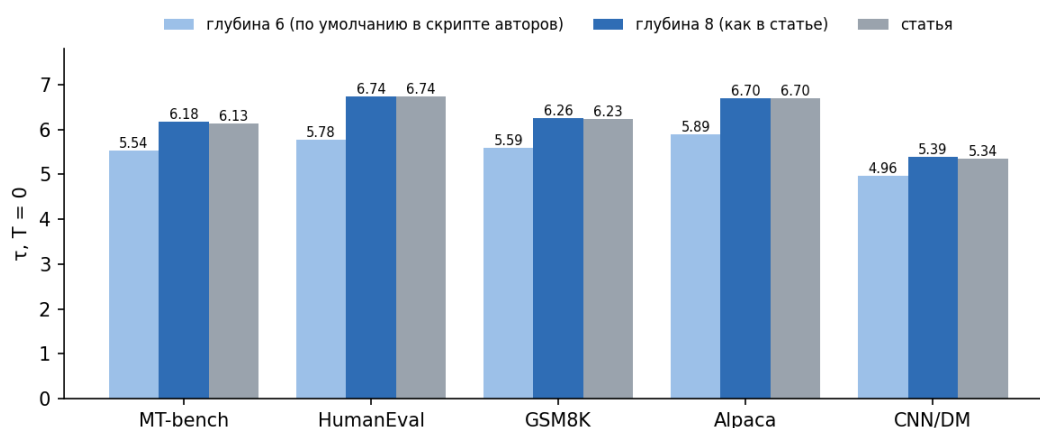


Рис. 4: τ при $T = 0$: глубина по умолчанию в скрипте авторов (6), глубина из статьи (8) и статья.

Найдено в коде авторов

- **Глубина дерева по умолчанию** в `gen_ea_answer_llama3chat.py` — `--depth 5`, это дерево EAGLE-2. Для EAGLE-3 нужно `--depth 7` (глубина 8 в статье: в коде глубина считается как длина ветки минус 1).
- `--max-new-token 1024` объявлен, но в генерацию не передаётся: фактический лимит ответа — 512 токенов.
- **Опубликованный тренер `traineagle3` падает при запуске:** `cnets.py` читает `self.train_config.gradient_checkpointing`, а `main.py` передаёт словарь. Исправление — две строки; обучить драфт их кодом без него нельзя.
- `speed.py` считает только ускорение (с жёстко прописанным токенизатором LLaMA-2); τ берём по определению статьи из счётчиков генератора.

4. Скорость шага обучения: код авторов против NeMo

Одни и те же 1000 примеров (PerfectBlend, ответы LLaMA-3.1-8B) и общий словарь драфта на 32K; замер по шагам оптимизатора после 20 шагов прогрева.

Запуск	Токенов/с	Относительно авторов
Код авторов как есть (fp16, gradient checkpointing, без паддинга, их расписание lr)	6 925	1,00×
NeMo, упаковка примеров в строки по 2048	7 326	1,06×
NeMo, + torch.compile	8 454	1,22×
NeMo, + torch.compile + FP8	8 577	1,24×
Одинаковая работа (BF16, каждый пример дополнен до 2048): авторы / NeMo	1 638 / 2 259	NeMo 1,38×

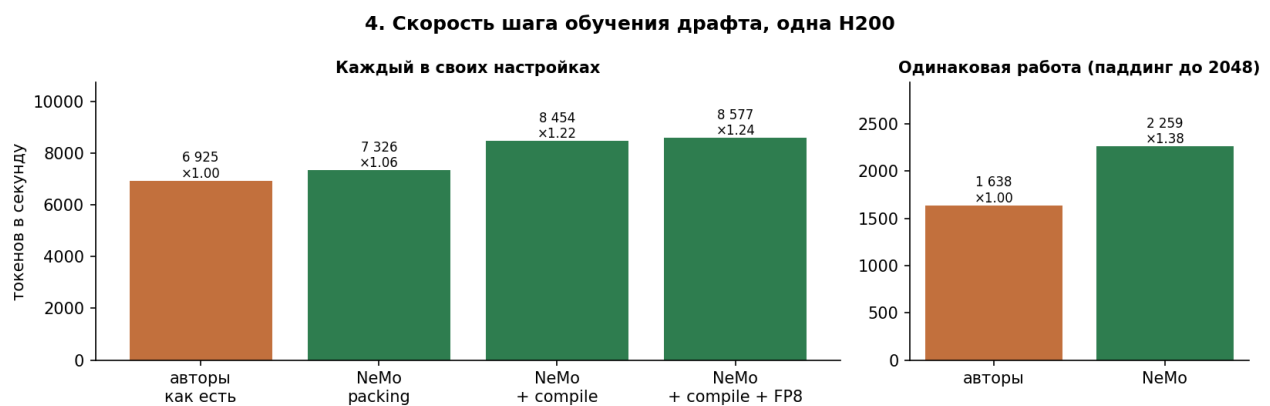


Рис. 5: Скорость шага обучения драфта на одной H200.

- **Выигрыш NeMo скромный — около 1,2×**. Код авторов не тратит время на паддинг (микробатч 1, каждый пример своей длины); главный приём NeMo — упаковка — даёт по сути то же.
- **torch.compile даёт +15 %, FP8 — ещё +1,5 %**: драфт — один слой, его матрицы слишком малы, чтобы FP8 окупился.
- **Память — 28–37 ГБ из 141 ГБ**: запас можно использовать для большего батча.
- **Оговорки**. Сравнивается скорость двух стеков, а не одинаковое обучение: разные версии torch (2.5.1 и 2.10), детали loss, RoPE драфта, точность оптимизатора. flash-attn 2 не проверен: на сервере нет CUDA toolkit.

Что дальше

- **3b, SGLang**: тот же драфт и вопросы, деревья 8/10/60 (как в статье), 3/1/4 (NeMo по умолчанию), 5/8/32, 6/10/60; T = 0 и T = 1 с тремя seed. Ожидаем, что CUDA graphs снимут накладные расходы драфта и ускорение приблизится к статье.
- **5, бюджет**: проба регенерации ShareGPT/UltraChat таргетом и расчёт GPU-часов. Предварительно: ≈8,5 тыс. токенов/с на H200, эпоха полного датасета (≈532 тыс. диалогов) — порядка 12 ч; 40 эпох из кода авторов в 100 ч не помещаются.

Код, все числа и протокол — в репозитории, папка [reproduction/](#).